

Conjuntos de instruções: modos e formatos de endereçamento

11.1 Endereçamento

- Endereçamento imediato
- Endereçamento direto
- Endereçamento indireto
- Endereçamento de registradores
- Endereçamento indireto por registradores
- Endereçamento por deslocamento
- Endereçamento de pilha

11.2 Modos de endereçamento x86 e ARM

- Modos de endereçamento x86
- Modos de endereçamento ARM

11.3 Formatos de instruções

- Tamanho da instrução
- Alocação de bits
- Instruções de tamanho variável

11.4 Formatos de instruções x86 e ARM

- Formatos de instruções x86
- Formatos de instruções ARM

11.5 Linguagem de montagem

11.6 Leitura recomendada

PRINCIPAIS PONTOS

- Uma referência a um operando dentro de uma instrução contém o valor atual do operando (imediato) ou uma referência para o endereço do operando. Uma grande variedade de modos de endereçamento é usada em vários conjuntos de instruções. Isso inclui modo direto (endereço do operando está no campo de endereço), modo indireto (o campo de endereço aponta para um local que contém o endereço do operando), modo de registrador, modo de registrador indireto e várias formas de deslocamento nos quais um valor de registrador é adicionado a um valor de endereço para produzir o endereço do operando.
- O formato da instrução define o layout dos campos dentro da instrução. A definição do formato da instrução é uma tarefa complexa que envolve considerações como tamanho da instrução, tamanho fixo ou variável, número de bits atribuído para *opcode* e para cada referência de operando e como o modo de endereçamento é definido.

No Capítulo 10 focamos *no que* um conjunto de instruções faz. Mais precisamente, examinamos os tipos de operandos e operações que podem ser especificados pelas instruções da máquina. Este capítulo se concentra na questão de *como* definir operandos e operações das instruções. Duas questões surgem. A primeira é como é especificado o endereço de um operando e a segunda, como são organizados os bits de uma instrução para definir o endereço do operando e a operação dessa instrução.



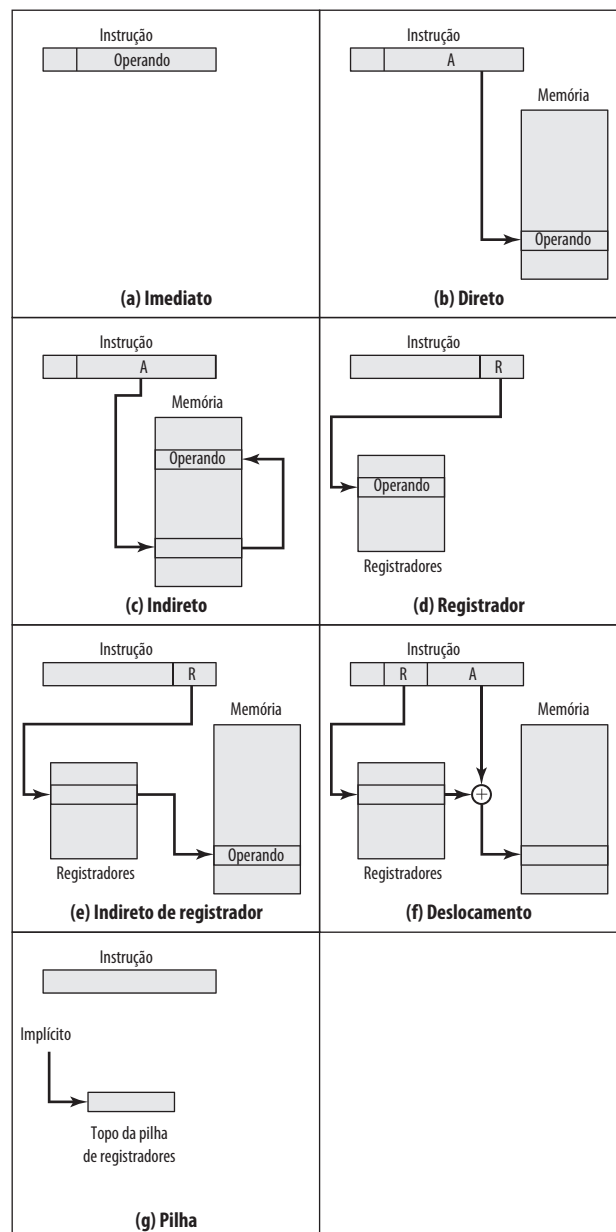
11.1 Endereçamento

O campo ou os campos de endereço num formato de instrução típico são relativamente pequenos. Nós gostaríamos de poder referenciar um grande intervalo de posições da memória principal ou, em alguns sistemas, da memória virtual. Para alcançar esse objetivo, uma grande variedade de técnicas de endereçamento foi desenvolvida. Todas envolvem algum tipo de troca entre intervalo de endereços e/ou flexibilidade de endereçamento por um lado e o número de referências de memória dentro da instrução e/ou a complexidade de cálculo de endereços por outro. Nesta seção, examinaremos as técnicas de endereçamento mais comuns:

- Imediato.
- Direto.
- Indireto.
- Registrador.
- Indireto por registrador.
- Deslocamento.
- Pilha.

Os modos são ilustrados na Figura 11.1. Nesta seção utilizamos a seguinte notação:
A = conteúdos de um campo de endereço dentro da instrução.

Figura 11.1 Modos de endereçamento



R = conteúdos de um campo de endereço dentro da instrução que se refere a um registrador.

EA = endereço real (efetivo) do local que contém o operando referenciado.

(X) = conteúdos do local de memória X ou do registrador X.

A Tabela 11.1 mostra o cálculo de endereço efetuado para cada modo de endereçamento.

Antes de iniciar esta discussão, dois comentários precisam ser feitos. Primeiro: a princípio, todas as arquiteturas de computadores oferecem mais do que um modo de endereçamento. A questão é como o processador pode determinar qual modo de endereçamento está sendo usado em uma determinada instrução. Existem diversas abordagens. Frequentemente, *opcodes* diferentes irão usar modos de endereçamento diferentes. Além disso, um ou mais bits dentro do formato da instrução podem ser usados como um *campo de modo*. O valor do campo de modo determina qual modo de endereçamento será usado.

O segundo comentário diz respeito à interpretação do endereço efetivo (EA). Em um sistema sem a memória virtual, o endereço efetivo será um endereço da memória principal ou um registrador. Em um sistema com memória virtual, o endereço efetivo é um endereço virtual ou um registrador. O mapeamento para um endereço físico é uma função da unidade de gerenciamento de memória (MMU — *memory management unit*) e é transparente para o programador.



Endereçamento imediato

A forma mais simples de endereçamento é o endereçamento imediato, no qual o valor do operando está presente na instrução

Operando = A

Este modo pode ser usado para definir e utilizar constantes ou definir valores iniciais das variáveis. Normalmente, o número será armazenado em duas formas complementares; o bit à esquerda do campo do operando é usado como bit de sinal. Quando o operando é carregado num registrador de dados, o bit de sinal é estendido para esquerda até o tamanho total da palavra de dados. Em alguns casos, o valor binário imediato é interpretado como um número inteiro e sem sinal.

A vantagem do endereçamento imediato é que nenhuma referência de memória, além de obter a instrução em si, é necessária para obter operando, economizando dessa forma um ciclo de memória ou cache dentro do ciclo da instrução. A desvantagem é que o tamanho do número é limitado ao tamanho do campo de endereço, o qual é, na maioria dos conjuntos de instruções, pequeno se comparado ao tamanho da palavra.



Endereçamento direto

Uma forma muito simples de endereçamento é o endereçamento direto, onde o campo de endereço contém o endereço efetivo do operando:

EA = A

Tabela 11.1 Modos básicos de endereçamento

Modo	Algoritmo	Principal vantagem	Principal desvantagem
Imediato	Operando = A	Nenhuma referência de memória	Magnitude de operando limitada
Direto	EA = A	Simples	Espaço de endereçamento limitado
Indireto	EA = (A)	Espaço de endereçamento grande	Múltiplas referências de memória
Registrador	EA = R	Nenhuma referência de memória	Espaço de endereçamento limitado
Indireto de registrador	EA = (R)	Espaço de endereçamento grande	Referência extra de memória
Deslocamento	EA = A + (R)	Flexibilidade	Complexidade
Pilha	EA = topo da pilha	Nenhuma referência de memória	Aplicabilidade limitada

A técnica era comum nas primeiras gerações dos computadores, porém não é comum em arquiteturas atuais. Ela requer apenas uma referência de memória e nenhum cálculo especial. A limitação óbvia é que ela oferece um espaço de endereços limitado.



Endereçamento indireto

No endereçamento direto, o tamanho do campo de endereço é normalmente menor do que o tamanho da palavra, limitando dessa forma o intervalo de endereços. Uma solução é ter um campo de endereço se referindo ao endereço de uma palavra na memória, o qual, por sua vez, contém o endereço completo do operando. Esta técnica é conhecida como **endereçamento indireto**:

$$EA = (A)$$

Os parênteses devem ser interpretados como *conteúdo de*. A vantagem óbvia desta abordagem é que, para o tamanho N de uma palavra, um espaço de endereçamento 2^N estará disponível. A desvantagem é que a execução da instrução requer duas referências de memória para obter o operando: um para obter o seu endereço e outra para obter o seu valor.

Embora o número de palavras que agora podem ser endereçadas seja igual a 2^N , o número de endereços efetivos diferentes que podem ser referenciados em qualquer momento é limitado a 2^K , onde K é o tamanho do campo de endereço. Normalmente isso não é um incômodo e pode até ser uma vantagem. Em um ambiente de memória virtual, todos os locais de endereços efetivos podem ser colocados na página 0 de qualquer processo. Como o campo de endereço de uma instrução é pequeno, ele irá naturalmente produzir endereços diretos de números pequenos, os quais aparecem na página 0. (A única restrição é que o tamanho da página deve ser igual ou maior que 2^K). Quando um processo está ativo, há referências repetidas na página 0, o que faz com que elas permaneçam na memória real. Assim, uma referência de memória indireta irá envolver, no máximo, uma falha de página em vez de duas.

Uma variação de endereçamento indireto raramente usada é o endereçamento indireto de vários níveis ou de cascata:

$$EA = (... (A) ...)$$

Neste caso, um bit de um endereço de palavra inteira é um flag indireto (I). Se o bit I é 0, então a palavra contém EA . Se o bit I é 1, então outro nível de indireção é usado. Aparentemente não há nenhuma vantagem nesta abordagem e a sua desvantagem é que três ou mais referências de memória podem ser necessárias para obter um operando.



Endereçamento de registradores

Endereçamento de registradores é semelhante ao endereçamento direto. A única diferença é que o campo de endereço se refere a um registrador em vez de um endereço da memória principal:

$$EA = R$$

Para esclarecer mais, se o conteúdo de um campo de endereço de registrador dentro de uma instrução for 5, então o registrador $R5$ é o endereço pretendido e o valor do operando está contido em $R5$. Normalmente, um campo de endereço que referencia registradores terá de 3 a 5 bits, então um total de 8 a 32 registradores de uso geral pode ser referenciado.

As vantagens de endereçamento de registradores são: (1) apenas um pequeno campo de endereço é necessário dentro da instrução, (2) nenhuma referência de memória que consome tempo é necessária. Conforme discutido no Capítulo 4, o tempo de acesso a de um registrador interno do processador é muito menor do que para um endereço da memória principal. A desvantagem do endereçamento de registradores é o espaço de endereçamento muito limitado.

Se o endereçamento de registradores for muito usado em um conjunto de instruções, isso implicará utilização pesada dos registradores do processador. Por causa do número muito limitado de registradores (se comparado ao número de endereços da memória principal), o seu uso desta maneira apenas faz sentido se forem utilizados eficientemente. Se cada operando for trazido para um registrador a partir da memória principal, usado uma vez e depois retornado à memória principal, então um passo intermediário desnecessário será introduzido. Se, por outro lado, o operando permanecer no registrador durante várias operações, então uma economia real será obtida. Um

exemplo é o resultado intermediário dentro de uma operação de cálculo. Para este caso, suponha que o algoritmo para multiplicar complementos de dois seja implementado via software. O local, dentro do fluxograma, chamado A (Figura 9.12) é referenciado muitas vezes e deve ser implementado preferencialmente em um registrador do que em um local da memória principal.

Cabe ao programador ou ao compilador decidir quais valores devem permanecer em registradores e quais devem ser armazenados na memória principal. A maioria dos processadores modernos implementa vários registradores de uso geral, colocando a responsabilidade de uma execução eficiente nas mãos de um programador de linguagem de montagem (por exemplo, um projetista de compiladores).



Endereçamento indireto por registradores

Assim como endereçamento de registradores é análogo ao endereçamento direto, endereçamento indireto por registradores é análogo ao endereçamento indireto. Em ambos os casos, a única diferença é se o campo de endereço referencia um local de memória ou um registrador. Assim temos para endereçamento indireto de registradores:

$$EA = (R)$$

As vantagens e as limitações do endereçamento indireto por registradores são basicamente as mesmas do endereçamento indireto. Em ambos os casos, a limitação do espaço de endereçamento (intervalo de endereços limitado) do campo de endereço é superada fazendo com que o campo se refira a um local de memória do tamanho de uma palavra contendo um endereço. Além disso, o endereçamento indireto por registradores utiliza uma referência de memória a menos do que o endereçamento indireto.



Endereçamento por deslocamento

Uma forma muito poderosa de endereçamento combina as capacidades do endereçamento direto e do endereçamento indireto de registradores. Ela é conhecida por vários nomes dependendo do contexto do seu uso, mas o mecanismo básico é o mesmo. Nós iremos chamá-la de *endereçamento por deslocamento*:

$$EA = A + (R)$$

O endereçamento por deslocamento requer que a instrução tenha dois campos de endereço, dos quais ao menos um é explícito. O valor contido em um campo de endereço (valor = A) é usado diretamente. O outro campo de endereço, ou uma referência implícita baseada em *opcode*, refere-se a um registrador cujos conteúdos são adicionados a A para produzir um endereço efetivo.

Iremos descrever três dos usos mais comuns do endereçamento por deslocamento:

- Endereçamento relativo.
- Endereçamento por registrador base.
- Indexação.

ENDEREÇAMENTO RELATIVO Para endereçamento relativo, também chamado de endereçamento PC-relativo, o registrador implicitamente referenciado é o contador do programa (PC). Ou seja, o endereço da próxima instrução é adicionado ao campo de endereço para produzir EA. Normalmente, o campo de endereço é tratado como um número complementar para esta operação. Dessa forma, o endereço efetivo é o deslocamento relativo ao endereço da instrução.

Endereçamento relativo explora o conceito de localidade que foi discutido nos capítulos 4 e 8. Se a maioria das referências de memória está relativamente próxima à instrução sendo executada, então o uso de endereçamento relativo economiza bits de endereço dentro da instrução.

ENDEREÇAMENTO POR REGISTRADOR BASE Para endereçamento baseado em registradores, temos a seguinte interpretação: o registrador base contém um endereço da memória principal e o campo de endereço contém um deslocamento (normalmente um número inteiro sem sinal) desse endereço. A referência registrador pode ser explícita ou implícita.

O endereçamento por registrador base também explora a posição das referências de memória. É um meio conveniente para implementar a segmentação, a qual foi discutida no Capítulo 8. Em algumas implementações, um único registrador de segmento é empregado e usado implicitamente. Em outras, o programador pode escolher um registrador para guardar o endereço base de um segmento e a instrução deve referenciá-lo explicitamente. Neste último caso, se o tamanho do campo de endereço é K e o número de possíveis registradores é N , então uma instrução pode referenciar qualquer uma de N áreas de 2^K palavras.

INDEXAÇÃO Para indexação, normalmente temos a seguinte interpretação: o campo de endereço referencia um endereço da memória principal e o registrador referenciado contém um deslocamento positivo desse endereço. Observe que este uso é exatamente o oposto da interpretação do endereçamento por registrador base. É claro que isto é mais do que apenas uma interpretação do usuário. Pelo fato de o campo de endereço ser considerado um endereço de memória na indexação, normalmente ele contém mais bits quando comparado a um campo de endereço de uma instrução com endereçamento por registrador base. Além disso, devemos observar que existem alguns refinamentos na indexação que não seriam úteis em um contexto por registrador base. Apesar disso, o método para calcular EA é o mesmo para endereçamento por registrador base e indexação e, em ambos os casos, a referência do registrador é algumas vezes explícita e algumas vezes implícita (para diferentes tipos de processadores).

Um uso importante da indexação é permitir um mecanismo eficiente para efetuar operações iterativas. Considere, por exemplo, uma lista de números armazenada iniciando na posição A. Suponha que queiramos adicionar 1 para cada elemento da lista. Precisamos obter cada valor, adicionar 1 a ele e armazená-lo de volta. A sequência de endereços efetivos que precisamos é A, A + 1, A + 2, ..., até a última posição da lista. Com indexação, isso é feito facilmente. O valor A é armazenado no campo de endereço da instrução e o registrador escolhido, chamado de *registrador indexador*, é inicializado com 0. Depois de cada operação, o registrador indexador é incrementado por 1.

Como os registradores indexadores são comumente usados para essas tarefas iterativas, é normal que haja necessidade de incrementar e decrementar o registrador indexador depois de cada referência a ele. Por ser uma operação tão comum, alguns sistemas farão isso automaticamente como sendo parte do mesmo ciclo de instrução. Isso é conhecido como *autoindexação*. Se determinados registradores forem usados exclusivamente como indexadores, então a autoindexação pode ser chamada implícita e automaticamente. Se registradores de uso geral forem usados, a operação de autoindexação pode precisar ser sinalizada por um bit dentro da instrução. A autoindexação com uso de incremento pode ser descrita da seguinte forma

$$EA = A + (R)$$

$$(R) \leftarrow (R) + 1$$

Em algumas máquinas, o endereçamento indireto e a indexação são oferecidos e é possível usar os dois dentro da mesma instrução. Existem duas possibilidades: a indexação é executada antes ou depois da indireção.

Se a indexação for executada depois da indireção, ela é chamada de *pós-indexação*:

$$EA = (A) + (R)$$

Primeiramente, os conteúdos dos campos de endereço são usados para acessar o local de memória contendo o endereço direto. Esse endereço é então indexado por um valor de registrador. Esta técnica é útil para acessar um dentro de vários blocos de dados de formato fixo. Por exemplo, foi descrito no Capítulo 8 que os sistemas operacionais precisam implementar um bloco para controle de processos para cada processo. A operação executada é a mesma, independentemente de qual bloco está sendo manipulado. Assim, os endereços nas instruções que referenciam o bloco poderiam apontar para um local (valor = A) contendo um ponteiro para o início do bloco de controle de processos. O registrador indexador contém o deslocamento dentro do bloco.

Com *pré-indexação*, a indexação é executada antes da indireção:

$$EA = (A + (R))$$

Um endereço é calculado da mesma maneira como na indexação simples. Neste caso, no entanto, o endereço calculado não contém o operando, mas o endereço do operando. Um exemplo do uso desta técnica é a construção de uma tabela com múltiplos endereços de desvios. Em um determinado ponto do programa, pode haver uma ramificação para uma série de posições diferentes dependendo da condição. Uma tabela de endereços pode ser definida com início em A. Usando indexação nesta tabela, a localização desejada pode ser encontrada.

Normalmente, um conjunto de instruções não irá incluir ambos os modos de pré-indexação e pós-indexação.



Endereçamento de pilha

O último modo de endereçamento que iremos considerar é endereçamento de pilha. Conforme definido no Apêndice 9A, uma pilha é um vetor linear de posições. Às vezes é chamada de *lista pushdown* ou *lista*

último-a-entrar-primeiro-a-sair (last-in). A pilha é um bloco reservado de posições. Itens são adicionados ao topo da pilha para que, a qualquer momento, o bloco esteja parcialmente preenchido. Associado à pilha, temos um ponteiro cujo valor é o endereço do topo da pilha. Alternativamente, dois elementos do topo podem estar nos registradores do processador. Nesse caso, o ponteiro da pilha referencia o terceiro elemento da pilha (Figura 10.14b). O ponteiro da pilha é mantido em um registrador. Assim, as referências das posições da pilha em memória são na verdade endereços indiretos dos registradores.

O modo de endereçamento de pilha é uma forma de endereçamento implícito. As instruções da máquina não precisam incluir uma referência de memória e sim operar no topo da pilha.

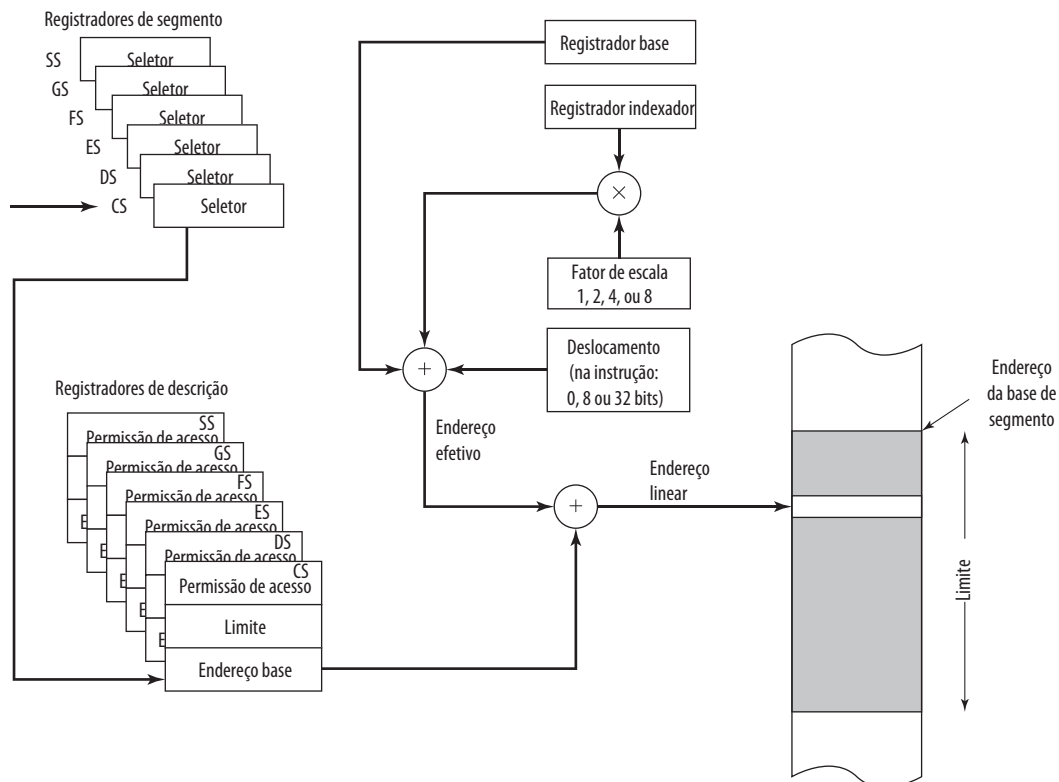
11.2 Modos de endereçamento x86 e ARM

Modos de endereçamento x86

Na Figura 8.21, o mecanismo de tradução de endereços x86 produz um endereço, chamado de endereço virtual ou efetivo, que é um offset dentro de um segmento. A soma do endereço inicial do segmento e do endereço efetivo produz um endereço linear. Se a paginação estiver sendo usada, este endereço linear tem que passar pelo mecanismo de tradução endereço de página para produzir um endereço físico. A seguir iremos ignorar este último passo porque ele é transparente para o conjunto de instruções e para o programador.

O x86 é equipado com uma série de formas de endereçamento que vieram possibilitar execução eficiente de linguagens de alto nível. A Figura 11.2 demonstra a lógica envolvida. O registrador de segmento determina o segmento que é objetivo da referência. Existem seis registradores de segmento; aquele usado para uma referência específica depende do contexto da execução e da instrução. Cada registrador de segmento

Figura 11.2 Cálculo do modo de endereçamento x86



mantém um índice da tabela de descritores de segmentos (Figura 8.20), a qual, por sua vez, mantém o endereço inicial dos segmentos correspondentes. Associado a cada registrador de segmento visível a usuário temos um registrador de descritor de segmento (não visível para programador), o qual grava as permissões de acesso para o segmento assim como o endereço inicial e o limite (tamanho) do segmento. Além disso, existem dois registradores que podem ser usados na construção do endereço: registrador básico e registrador indexador.

A Tabela 11.2 mostra os modos de endereçamento x86. Vamos analisar cada um deles.

No **modo imediato**, o operando está incluído na instrução. O operando pode ser um byte, uma palavra ou uma palavra dupla de dados.

No **modo em registrador operando**, o operando está localizado em um registrador. Para instruções gerais, tais como transferência de dados e instruções aritméticas e lógicas, o operando pode ser um dos registradores gerais de 32 bits (EAX, EBX, ECX, EDX, ESI, EDI, ESP, EBP), um dos registradores gerais de 16 bits (AX, BX, CX, DX, SI, DI, SP, BP) ou um dos registradores gerais de 8 bits (AH, BH, CH, DH, AL, BL, CL, DL). Existem também algumas instruções que referenciam registradores selecionadores de segmentos (CS, DS, ES, SS, FS, GS).

Os modos de endereçamento restantes referenciam posições da memória. A posição da memória precisa estar especificada em termos do segmento que contém a posição e offset do começo do segmento. Em alguns casos, o segmento é especificado explicitamente; em outros, ele é especificado pelas regras simples que definem um segmento padrão.

No **modo de deslocamento**, offset do operando (o endereço efetivo da Figura 11.2) é mantido como parte da instrução sendo um deslocamento de 8, 16 ou 32 bits. Com segmentação, todos os endereços das instruções se referem meramente a um offset dentro de um segmento. O modo de endereçamento por deslocamento é encontrado em poucas máquinas porque, conforme mencionado anteriormente, leva a instruções longas. No caso de x86, o valor de deslocamento pode ser de 32 bits, gerando uma instrução de 6 bytes. Endereçamento por deslocamento pode ser útil para referenciais variáveis globais.

Os modos de endereçamento restantes são indiretos, porque a parte da instrução que se refere ao endereço diz ao processador onde procurar pelo endereço. O **modo base** especifica que um dos registradores de 8, 16 ou 32 bits contém o endereço efetivo. Isto é equivalente ao que definimos como endereçamento indireto de registradores.

Tabela 11.2 Modos de endereçamento x86

Modo	Algoritmo
Imediato	$\text{Operando} = A$
Operando em registrador	$LA = R$
Deslocamento	$LA = (SR) + A$
Base	$LA = (SR) + B$
Base com deslocamento	$LA = (SR) + (B) + A$
Índice escalado com deslocamento	$LA = (SR) + (I) \times S + A$
Base com índice e deslocamento	$LA = (SR) + (B) + (I) + A$
Base com índice escalado e deslocamento	$LA = (SR) + (I) \times S + (B) + A$
Relativo	$LA = (PC) + A$

LA = endereço linear (*linear address*)

(X) = conteúdo de X

SR = registrador de segmento (*segment register*)

PC = contador de programador

A = conteúdo de um campo de endereço da instrução

R = registrador

B = registrador base

I = registrador indexado

S = fator de escalar

No **modo base com deslocamento**, as instruções incluem um deslocamento a ser adicionado ao registrador base, o qual pode ser qualquer um dos registradores de uso geral. Seguem os exemplos de uso deste modo:

- Usado pelo compilador para apontar o início de uma área de variável local. Por exemplo, o registrador básico pode indicar o início de um *stackframe*, a qual contém as variáveis locais para o procedimento correspondente.
- Usado para indexar um vetor quando o número de elementos não for 1, 2, 4 ou 8 bytes e por isso não puder ser indexado usando um registrador indexador. Neste caso, o deslocamento aponta para o começo do vetor e o registrador base mantém o resultado do cálculo para determinar o offset até um determinado elemento dentro do vetor.
- Usado para acessar um campo de um registro. O registrador base aponta para o início do registro, enquanto o deslocamento é um offset até o campo.

No **modo de índice escalado com deslocamento**, a instrução inclui um deslocamento a ser adicionado a um registrador, chamado neste caso de registrador indexador. O registrador indexador pode ser qualquer um dos registradores de uso geral exceto aquele chamado de ESP, o qual é normalmente usado para processamento de pilhas. Ao calcular o endereço efetivo, o conteúdo do registrador indexador é multiplicado por um fator escalar de 1, 2, 4 ou 8 e depois adicionado ao deslocamento. Este modo é muito conveniente para indexar vetores. O fator escalar de 2 pode ser usado para um vetor de inteiros de 16 bits. O fator escalar de 4 pode ser usado para inteiros de 32 bits ou números de ponto flutuante. Finalmente, o fator escalar de 8 pode ser usado para um vetor números de ponto flutuante de precisão dupla.

O **modo base com índice e deslocamento** soma os conteúdos do registrador base, registrador indexador e deslocamento para formar um endereço efetivo. Novamente, o registrador base pode ser qualquer um dos registradores de uso geral, exceto ESP. Por exemplo, este modo de endereçamento pode ser usado para acessar um vetor local dentro de uma estrutura de pilha. Ele também pode ser usado para suportar vetores bidimensionais; neste caso o deslocamento aponta para o início do vetor e cada registrador lida com uma dimensão do vetor.

O **modo base com índice escalado e deslocamento** soma o conteúdo do registrador indexador multiplicado por um fator escalar, o conteúdo do registrador base e deslocamento. Isso é útil se um vetor está armazenado dentro de uma pilha; neste caso, os elementos do vetor teriam tamanho de 2, 4 ou 8 bytes cada. Este modo fornece também indexação eficiente de um vetor bidimensional quando os elementos do vetor têm tamanho de 2, 4 ou 8 bytes.

Finalmente, o **modo relativo** pode ser usado em instruções de transferência de controle. Um deslocamento é adicionado ao valor do contador de programa, o qual aponta para a próxima instrução. Neste caso, o deslocamento é tratado como um valor com sinal de um byte, palavra ou palavra dupla e esse valor incrementa ou decrementa o endereço dentro do contador de programa.



Modos de endereçamento ARM

Normalmente, uma máquina RISC, diferentemente de uma máquina CISC, usa um conjunto de modos de endereçamento relativamente simples e direto. A arquitetura ARM difere um tanto dessa tradição ao prover um conjunto de modos de endereçamento relativamente rico. A forma mais conveniente de classificar esses modos é de acordo com o tipo da instrução.¹

ENDEREÇAMENTO DE CARGA/ARMAZENAMENTO (LOAD/STORE) Instruções para carregar e armazenar são as únicas instruções que referenciam a memória. Isso é feito sempre indiretamente através de um registrador básico e um offset. Existem três alternativas no que diz respeito à indexação (Figura 11.3):

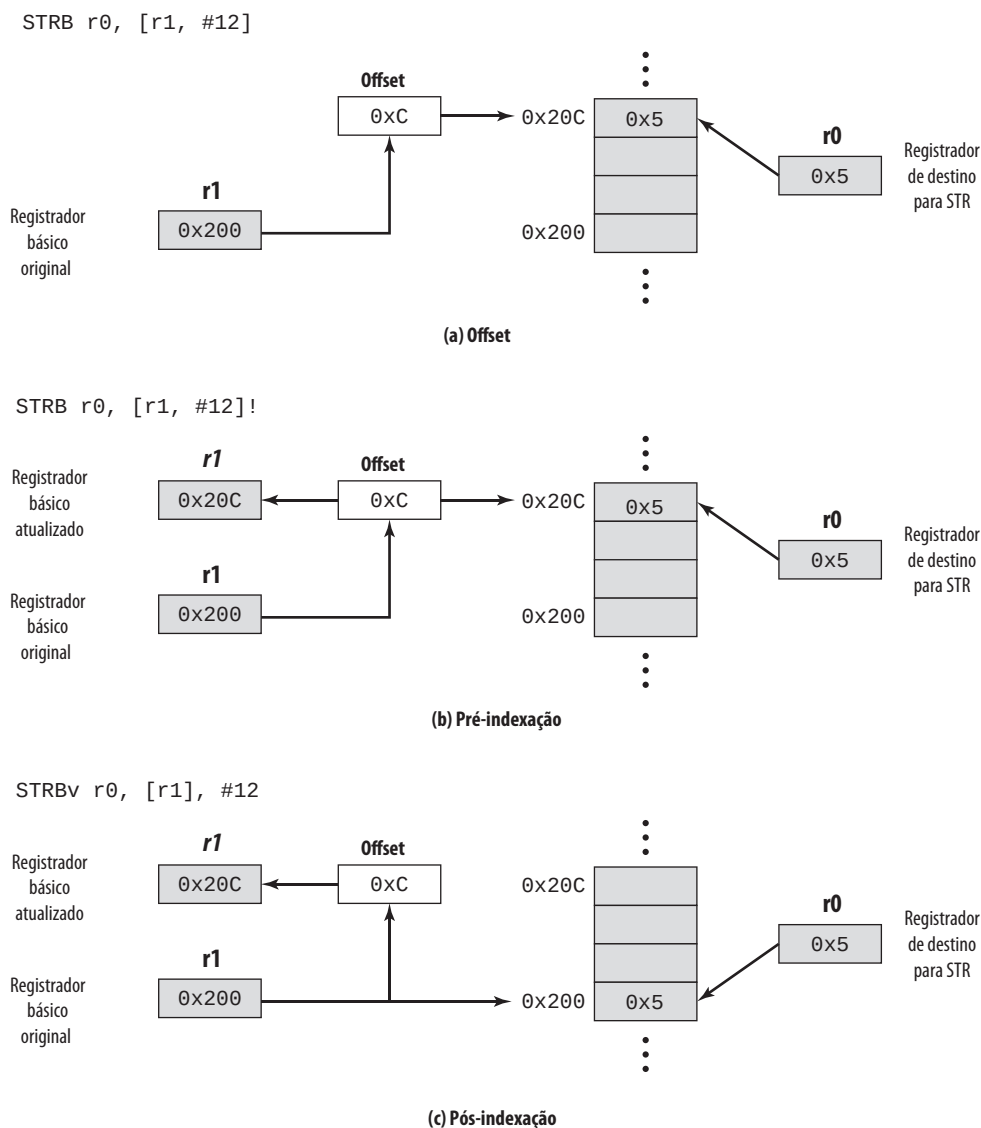
- **Offset:** para este modo de endereçamento, a indexação não é usada. Um valor de offset é adicionado ou subtraído do valor que está no registrador base para formar o endereço de memória. Como um exemplo, a Figura 11.3a ilustra esse método com a instrução `STRB r0, [r1, #12]` da linguagem de montagem. Esta é a instrução para armazenar byte. Neste caso, o endereço base está no registrador `r1` e o deslocamento é um valor imediato 12 decimal. O endereço resultante (base mais offset) é a posição onde o byte menos significativo de `r0` será armazenado.

¹ De acordo com a nossa discussão sobre endereçamento x86, ignoramos a tradução do endereço virtual para o físico durante a discussão a seguir.

- **Pré-indexação:** o endereço de memória é formado da mesma maneira como no endereçamento de offset. O endereço de memória é também armazenado de volta no registrador base. Em outras palavras, o valor do registrador base é incrementado ou decrementado pelo valor do offset. A Figura 11.3b ilustra este método com a instrução `STRB r0, [r1, #12]!` da linguagem de montagem. O ponto de exclamação significa pré-indexação.
- **Pós-indexação:** o endereço de memória é o valor do registrador base. Um offset é adicionado ou subtraído do valor do registrador base e o resultado é armazenado de volta no registrador base. A Figura 11.3c ilustra este método com a instrução `STRBv r0, [r1], #12` da linguagem de montagem.

Observe que aquilo a que ARM se refere como registrador base atua, na verdade, como um registrador indexador para endereçamento de pré-indexação e pós-indexação. O valor de offset pode ser ou um valor imediato armazenado na instrução ou pode estar em outro registrador. Se o valor de offset estiver em um registrador, outro recurso útil estará disponível: endereçamento de registrador escalado. O valor no registrador de offset é escalado por um dos operadores de deslocamento: operador de deslocamento lógico esquerdo, operador de deslocamento

Figura 11.3 Métodos de indexação ARM



lógico direito, operador de deslocamento aritmético direito, operador rotacional direito ou operador rotacional direito estendido (o qual inclui bit de carry na rotação). A quantidade de deslocamento é especificada como um valor imediato na instrução.

ENDEREÇAMENTO DE INSTRUÇÕES DE PROCESSAMENTO DE DADOS As instruções de processamento de dados usam endereçamento de registradores ou uma mistura de endereçamento de registradores e endereçamento imediato. Para endereçamento de registradores, o valor em um dos operandos do registrador pode ser escalado usando um dos cinco operadores de deslocamento definidos no parágrafo anterior.

INSTRUÇÕES DE DESVIOS A única forma de endereçamento para instruções de desvios é o endereçamento imediato. A instrução de desvio contém um valor de 24 bits. Para calcular o endereço, o valor é deslocado à esquerda por 2 bits para que o endereço esteja dentro do limite de uma palavra. Por isso, o intervalo de endereços efetivos é de mais ou menos 32 MB a partir do contador de programa.

ENDEREÇAMENTO DE CARGA/ARMAZENAMENTO MÚLTIPLO Instruções de carga múltipla carregam dados da memória para um subconjunto (possivelmente todos) de registradores de uso geral. Instruções de armazenamento múltiplo armazenam dados um subconjunto (possivelmente todos) de registradores de uso geral na memória. A lista de registradores para carregar ou armazenar é especificada dentro de um campo de 16 bits na instrução onde cada bit corresponde a um dos 16 registradores. Os modos de endereçamento de carga e armazenamento múltiplo produzem um intervalo sequencial de endereços de memória. O registrador de número mais baixo é armazenado no endereço de memória mais baixo e o registrador de número mais alto no endereço de memória mais alto. Quatro modos de endereçamento são usados (Figura 11.4): incremento depois, incremento antes, decremento depois e decremento antes. Um registrador básico especifica um endereço da memória principal onde os valores dos registradores são armazenados ou carregados em posições ascendentes (incremento) ou descendentes (decremento). O incremento ou decremento inicia ou antes ou depois do primeiro acesso à memória.

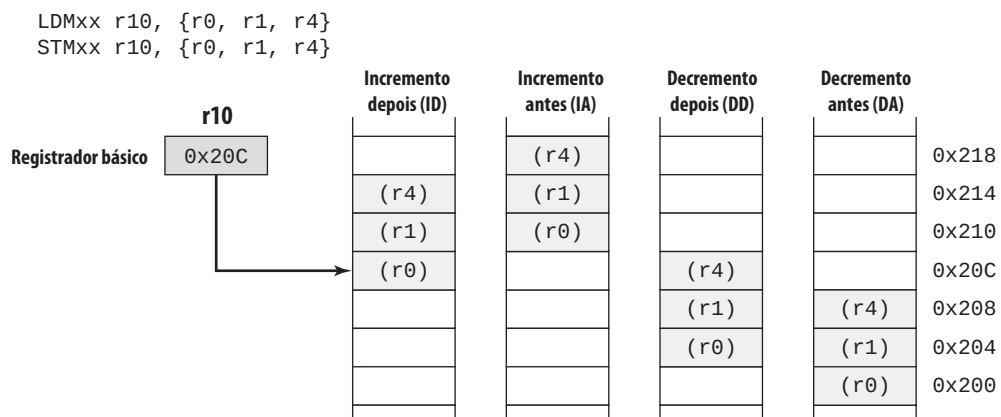
Essas instruções são úteis para bloqueios de leitura ou escrita, operações de pilha e sequências de saída dos procedimentos.



11.3 Formatos de instruções

Um formato da instrução define o layout de bits de uma instrução, no que diz respeito aos campos que a constituem. Um formato da instrução tem que incluir um *opcode* e, implícita ou explicitamente, zero ou mais operandos. Cada operando explícito é referenciado usando um dos modos de endereçamento descritos na Seção 11.1. O formato deve, implícita ou explicitamente, indicar o modo de endereçamento para cada operando. Na maioria dos conjuntos de instruções, mais do que um formato de instrução é usado.

Figura 11.4 Endereçamento de carga/armazenamento múltiplo



O projeto de um formato da instrução é uma arte complexa e uma variedade impressionante de designs têm sido implementados. Analisaremos os principais pontos de projeto, observando alguns rapidamente para ilustrar esses pontos, e depois analisaremos as soluções x86 e ARM em detalhes.



Tamanho da instrução

A questão mais básica enfrentada durante o projeto é o tamanho do formato da instrução. Esta decisão afeta, e é afetada pelo, tamanho da memória, organização da memória, estrutura do barramento, complexidade e velocidade do processador. Esta decisão determina a riqueza e a flexibilidade da máquina do ponto de vista do programador da linguagem de montagem.

A troca mais óbvia aqui está entre o desejo por um conjunto poderoso de instruções e a necessidade de economizar o espaço. Programadores querem mais *opcodes*, mais operandos, mais modos de endereçamento e maior intervalo de endereços. Mais *opcodes* e mais operandos tornam a vida do programador mais fácil, porque programas mais curtos podem ser escritos para executar uma determinada tarefa. Da mesma forma, mais modos de endereçamento dão maior flexibilidade ao programador para implementar certas funções, tais como manipulação de tabelas e múltiplos destinos de desvios. E, obviamente, com o aumento do tamanho da memória principal e com o uso crescente da memória virtual, os programadores querem poder endereçar intervalos de memória maiores. Todas estas coisas (*opcodes*, operandos, modos de endereçamento, intervalos de endereços) requerem bits e levam a tamanhos maiores das instruções. Mas tamanhos maiores das instruções podem ser um desperdício. Uma instrução de 64 bits ocupa o dobro de espaço de uma instrução de 32 bits, mas provavelmente não é duas vezes mais útil.

Por trás desta negociação básica, existem outras considerações. Ou o tamanho da instrução deveria ser igual ao tamanho transferência de memória (em um sistema de barramento isso seria o tamanho da informação do barramento) ou um deveria ser múltiplo do outro. Caso contrário, não teremos um número inteiro de instruções durante um ciclo de busca. Uma consideração relacionada é a taxa de transferência da memória. Esta taxa não acompanhou o aumento na velocidade dos processadores. Assim, a memória pode se tornar um gargalo se o processador puder executar as instruções mais rapidamente do que as obtém. Uma solução para este problema é usar memória cache (veja a Seção 4.3); outra é usar instruções mais curtas. Desta forma, uma instrução de 16 bits pode ser obtida em uma taxa duas vezes maior do que uma instrução de 32 bits, mas provavelmente pode ser executada numa velocidade menos que duas vezes mais.

Um recurso aparentemente mais comum, mas não menos importante, é que o tamanho da instrução deve ser um múltiplo do tamanho de um caractere, o que é normalmente de 8 bits, e do tamanho de números de ponto fixo. Para perceber isso, precisamos fazer o uso da definição infeliz da palavra, *palavra* (Frailey, 1983^a). O tamanho de memória de uma palavra é, de uma certa forma, uma unidade “natural” de organização. O tamanho de uma palavra normalmente determina o tamanho dos números de ponto fixo (normalmente os dois são iguais). O tamanho de uma palavra é também comumente igual a, ou pelo menos totalmente relacionado ao, tamanho de transferência da memória. Como uma forma comum de dado é o caractere, gostaríamos que uma palavra armazenasse um número inteiro de caracteres. Caso contrário, haverá bits desperdiçados em cada palavra quando múltiplos caracteres forem armazenados ou um caractere terá que ultrapassar o limite de uma palavra. A importância deste ponto é tanta que a IBM, quando introduziu System/360 e quis empregar caracteres de 8 bits, tomou a radical decisão de mudar da arquitetura de 36 bits das séries 700/7000 para uma arquitetura de 32 bits.



Alocação de bits

Analisamos alguns dos fatores que devem ser levados em consideração na hora de decidir o tamanho do formato instrução. Uma questão igualmente difícil é como alocar os bits nesse formato. As negociações aqui são complexas.

Para um determinado tamanho da instrução, existe uma negociação clara entre o número de *opcodes* e da capacidade de endereçamento. Mais *opcodes* obviamente significam mais bits no campo de *opcode*. Para um formato da instrução de um determinado tamanho, isso reduz o número de bits disponíveis para endereçamento. Existe um refinamento interessante nesta negociação: o uso de *opcodes* de tamanho variável. Nesta abordagem, existe um tamanho mínimo de *opcode*, mas, para alguns *opcodes*, operações adicionais podem ser especificadas com uso de

bits adicionais na instrução. Para uma instrução de tamanho fixo, isso deixa menos bits para endereçamento. Por isso este recurso é usado para aquelas instruções que requerem menos operandos e/ou endereçamento menos poderoso.

Os seguintes fatores relacionados são importantes para determinar o uso de bits de endereçamento.

- **Número de modos de endereçamento:** às vezes um modo de endereçamento pode ser indicado implicitamente. Por exemplo, alguns *opcodes* podem sempre chamar a indexação. Em outros casos, os modos de endereçamento devem ser explícitos e um ou mais bits de modo serão necessários.
- **Número de operandos:** vimos que menos endereços podem produzir programas mais longos e estranhos (Figura 10.3). Instruções comuns em máquinas atuais fornecem dois operandos. Cada endereço de operando na instrução pode requerer seu próprio indicador de modo ou o uso de um indicador de modo pode ser limitado a apenas um dos campos de endereço.
- **Registrador versus memória:** uma máquina tem que ter registradores para que os dados possam ser trazidos dentro do processador para serem processados. Com apenas um único registrador visível ao usuário (normalmente chamado de acumulador), um endereço de operando é implícito e não consome nenhum bit da instrução. No entanto, programação com um único registrador é estranha e requer muitas instruções. Mesmo com vários registradores, apenas alguns poucos bits são necessários para especificar o registrador. Quanto mais esses registradores puderem ser usados para referenciar operandos, menos bits serão necessários. Uma série de estudos indica que um total de 8 a 32 registradores é desejável (Lunde, 1977^b, Huck, 1983^c). A maioria das arquiteturas atuais possui pelo menos 32 registradores.
- **Número de conjuntos de registradores:** a maioria das máquinas atuais possui um conjunto de registradores de uso geral, normalmente com 32 ou mais registradores dentro do conjunto. Esses registradores podem ser usados para armazenar dados e para armazenar endereços para endereçamento por deslocamento. Algumas arquiteturas, incluindo a x86, possuem dois ou mais conjuntos especializados (como dados e deslocamento). Uma vantagem desta última abordagem é que, para um número fixo de registradores, uma divisão funcional requer menos bits para serem usados na instrução. Por exemplo, com dois conjuntos de oito registradores, apenas 3 bits são requeridos para identificar um registrador; o *opcode* ou registrador de modo irá determinar qual conjunto dos registradores está sendo referenciado.
- **Intervalo de endereços:** para endereços que referenciam a memória, o intervalo de endereços que pode ser referenciado está relacionado com o número de bits de endereço. Como isso impõe uma limitação séria, endereçamento direto é raramente usado. Com endereçamento por deslocamento, o intervalo está aberto até o tamanho do registrador de endereço. Mesmo assim, ainda é conveniente permitir deslocamentos grandes a partir do endereço no registrador, o que requer um número de bits de endereço relativamente grande dentro da instrução.
- **Granularidade do endereço:** para endereços que referenciam memória no lugar de registradores, outro fator é a granularidade do endereçamento. Em um sistema com palavras de 16 ou 32 bits, um endereço pode referenciar uma palavra ou um byte de acordo com a preferência do projeto. Endereçamento de bytes é conveniente para manipulação de caracteres, porém exige mais bits de endereço para um tamanho fixo de memória.

Assim, a pessoa responsável pelo projeto tem que considerar e equilibrar diversos fatores. Não está claro o quão críticas são algumas dessas escolhas. Como exemplo podemos citar um estudo (Cragon, 1979^d) que comparou várias abordagens de formatos das instruções, inclusive o uso de pilha, registradores de uso geral, um acumulador e abordagens exclusivamente memória-para-registrador. Usando um conjunto consistente de hipóteses, nenhuma diferença significativo foi observada no espaço de código ou tempo de execução.

Vamos observar rapidamente como dois projetos históricos de máquinas equilibram esses vários fatores.

PDP-8 Um dos projetos de instrução mais simples para um computador de uso geral foi o feito para PDP-8 (Bell, Newell e Siewiorek, 1978^e). PDP-8 usa instruções de 12 bits e trata palavras de 12 bits. Existe um único registrador de uso geral, o acumulador.

Apesar das limitações deste projeto, o endereçamento é bastante flexível. Cada referência de memória consiste de 7 bits mais dois modificadores de 1 bit. A memória é dividida em páginas de tamanho fixo de $2^7 = 128$ palavras cada. O cálculo de endereços é baseado em referências à página 0 ou página corrente (página contendo esta instrução) conforme determinado pelo bit de página. O segundo bit modificador indica se endereçamento direto ou indireto

está sendo usado. Estes dois modos podem ser usados em conjunto, então um endereço indireto é um endereço de 12 bits contido em uma palavra da página 0 ou na página corrente. Além disso, 8 palavras dedicadas na página 0 são “registradores” autoindexados. Quando uma referência indireta é feita a uma dessas posições, ocorre a pré-indexação.

A Figura 11.5 mostra o formato da instrução PDP-8. Existe um *opcode* de 3 bits e três tipos de instruções. Para *opcodes* de 0 a 5, o formato é uma instrução de referência à memória de endereço único incluindo um bit de página e um bit indireto. Assim, existem apenas seis operações básicas. Para aumentar o grupo de operações, *opcode* 7 define uma referência de registrador ou *microinstrução*. Neste formato, os bits restantes são usados para codificar operações adicionais. Em geral, cada bit define uma operação específica (por exemplo, limpar o acumulador) e esses bits podem ser combinados em uma única instrução. A estratégia de microinstrução tem sido usada desde o antigo PDP-1 e é, de certa forma, o precursor das máquinas microprogramadas de hoje, o que será discutido na Parte 4. *Opcode* 6 é operação de E/S; 6 bits são usados para selecionar um dos 64 dispositivos e 3 bits especificam um comando de E/S em particular.

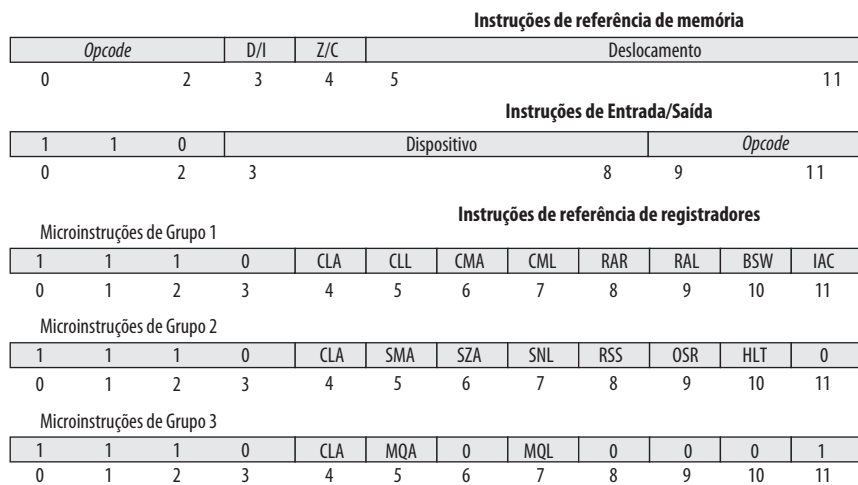
O formato da instrução PDP-8 é incrivelmente eficiente. Ele suporta endereçamento indireto, endereçamento por deslocamento e indexação. Com uso da extensão de *opcode*, ele suporta um total de aproximadamente 35 instruções. Dada a limitação do tamanho da instrução de 12 bits, os projetistas dificilmente poderiam ter feito algo melhor.

PDP-10 Um grande contraste ao conjunto de instruções PDP-8 é o PDP-10. Ele foi projetado para ser um sistema de tempo compartilhado em grande escala, com ênfase em ser fácil de programar, mesmo que hardware adicional fosse envolvido.

Dentre os princípios empregados ao se projetar o conjunto de instruções, estão os seguintes (Bell, 1978^f):

- **Ortogonalidade:** ortogonalidade é o princípio onde duas variáveis são independentes uma da outra. No contexto de um conjunto de instruções, o termo significa que outros elementos de uma instrução são independentes do (não determinados por) *opcode*. Os projetistas de PDP-10 usam o termo para descrever o fato de que um endereço é computado sempre do mesmo jeito, independentemente da *opcode*. Isto é diferente de muitas máquinas onde o modo de endereçamento às vezes depende implicitamente do operador que está sendo usado.

Figura 11.5 Formato da Instrução de PDP-8



D/I = endereço Direto/Indireto

Z/C = página 0 ou atual

CLA = limpar acumulador

CLL = limpar link

CMA = acumulador de complemento

CML = link de complemento

RAR = rotacionar acumulador para direita

RAT = rotacionar acumulador para esquerda

BSW = trocar byte

IAC = incrementar acumulador

SMA = pular quando acumulador negativo

SZA = pular quando acumulador zero

SNL = pular quando link não é zero

RSS = reverter sentido quando pular

OSR = ou com troca de registrador

HLT = parar

MQA = quociente múltiplo no registrador

ML = carregar quociente múltiplo

- **Integridade:** cada tipo aritmético de dados (inteiro, ponto fixo, ponto flutuante) deve ter um conjunto de operações completo e idêntico.
- **Endereçamento direto:** endereçamento básico com deslocamento, o qual deixa a organização da memória a cargo do programador, foi substituído pelo endereçamento direto.

Cada um destes princípios colabora em atingir o objetivo principal que é a facilidade de programação.

O PDP-10 possui uma palavra de 36 bits e uma instrução de 36 bits. O formato fixo da instrução é mostrado na Figura 11.6. O *opcode* ocupa 9 bits, permitindo até 512 operações. Na verdade, um total de 365 instruções foi definido. A maioria das instruções possui dois endereços, um dos quais é um dos 16 registradores de uso geral. Desta forma, a referência deste operando ocupa 4 bits. A referência do outro operando começa com um campo de endereço de memória de 18 bits, que pode ser usado como um operando imediato ou um endereço de memória. No último uso, são permitidos endereçamento indireto e indexação. Os mesmos registradores de uso geral são usados também como registradores indexadores.

Uma instrução com tamanho de 36 bits é um verdadeiro luxo. Não há necessidade de inventar coisas esperas para obter mais *opcodes*; um campo de *opcode* de 9 bits é mais do que suficiente. O endereçamento também é direto. Um campo de endereço de 18 bits torna endereçamento direto desejável. Indireção é fornecida para tamanhos de memória maiores que 2^{18} . Para facilitar a programação, é fornecida a indexação para manipular tabelas e para programas iterativos. Além disso, com um campo de operando de 18 bits, o endereçamento imediato se torna atraente.

O projeto do conjunto de instruções PDP-10 atinge os objetivos mencionados anteriormente (Lunde, 1977^b). Ele facilita a tarefa do programador ou compilador com o custo de uma utilização de espaço ineficiente. Essa foi uma escolha consciente tomada por projetistas e, portanto, não pode ser considerada como uma falha no projeto.



Instruções de tamanho variável

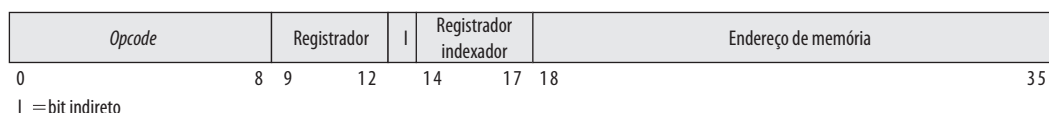
Os exemplos que vimos até agora usam um único tamanho fixo de instrução e discutimos as negociações implicitamente nesse contexto. Mas, em vez disso, os projetistas poderiam ter escolhido oferecer uma variedade de formatos de instruções de tamanhos diferentes. Esta tática torna fácil fornecer um grande repertório de *opcodes* com tamanhos diferentes. O endereçamento pode ser mais flexível com várias combinações de referências a registradores e à memória e com modos de endereçamento. Com instruções de tamanho variável, estas variações podem ser fornecidas de uma forma eficiente e compacta.

O principal preço a ser pago pelas instruções de tamanho variável é o aumento na complexidade do processador. A queda nos preços de hardware, o uso de microprogramação (discutida na Parte 4) e um aumento geral no entendimento dos princípios de projeto dos processadores contribuíram para que esse preço se tornasse pequeno. No entanto, veremos que as máquinas RISC e as superescaláveis podem explorar o uso de instruções de tamanho fixo para melhorar desempenho.

O uso de instruções de tamanho variável não acaba com a necessidade de relacionar integralmente todos os tamanhos das instruções com o tamanho da palavra. Como o processador não conhece o tamanho da próxima instrução a ser obtida, uma estratégia comum é buscar o número de bytes ou palavras pelo menos igual ao tamanho da maior instrução possível. Isso significa que, às vezes, várias instruções são lidas. No entanto, como veremos no Capítulo 12, esta é uma boa estratégia a ser seguida em todos os casos.

PDP-11 O PDP-11 foi projetado para oferecer um conjunto de instruções poderoso e flexível dentro das limitações de um minicomputador de 16 bits (Bell *et al.* 1970^g).

Figura 11.6 Formato da instrução PDP-10



O PDP-11 utiliza um conjunto de oito registradores de uso geral de 16 bits. Dois desses registradores têm uma importância adicional: um é usado como ponteiro de pilha para operações especiais de pilha e outro é usado como contador de programa, o qual contém o endereço da próxima instrução.

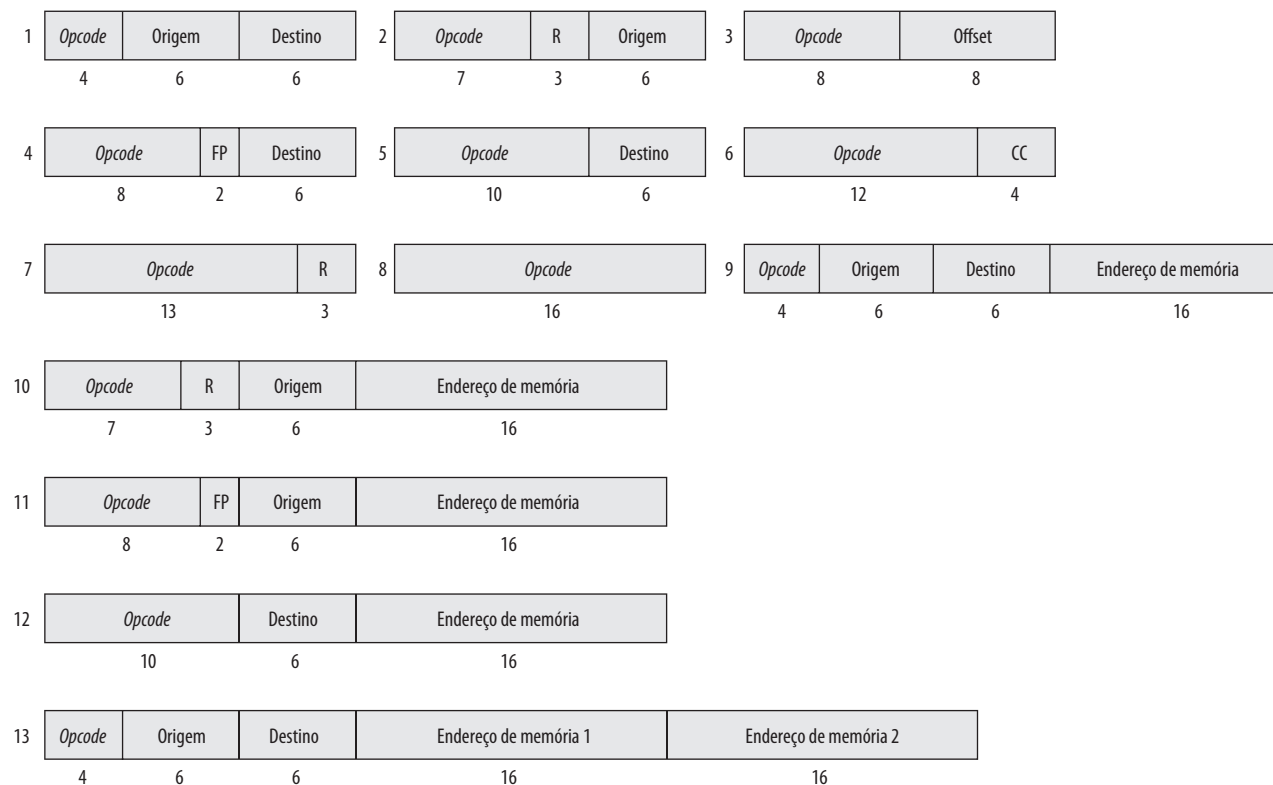
A Figura 11.7 mostra os formatos da instrução do PDP-11. Treze formatos diferentes são usados, incluindo tipos de instrução de zero, um e dois endereços. O *opcode* pode variar de tamanho de 4 a 16 bits. Referências à registradores têm o tamanho de 6 bits. Três bits identificam o registrador e 3 bits restantes identificam o modo de endereçamento. O PDP-11 possui um conjunto rico de modos de endereçamento. Uma vantagem de ligar o modo de endereçamento ao operando em vez do *opcode*, como é feito às vezes, é que qualquer modo de endereçamento pode ser usado com qualquer *opcode*. Conforme mencionamos, esta independência é conhecida como *ortogonalidade*.

As instruções PDP-11 normalmente têm o tamanho de uma palavra (16 bits). Para algumas instruções, um ou dois endereços de memória são adicionados para que instruções de 32 e 42 bits estejam disponíveis. Isso possibilita maior flexibilidade no endereçamento.

O conjunto de instruções e a capacidade de endereçamento do PDP-11 é complexo. Isso aumenta o custo de hardware e a complexidade de programação. A vantagem é que programas mais eficientes e compactos podem ser desenvolvidos.

VAX A maioria das arquiteturas fornece um número relativamente pequeno de formatos fixos de instruções. Isso pode causar dois problemas para os programadores. Primeiro, o modo de endereçamento e *opcode* não são ortogonais. Por exemplo, para uma certa operação, um operando deve vir a partir de um registrador e outro da memória, ou ambos dos registradores e assim por diante. Segundo, apenas um número limitado de operandos pode

Figura 11.7 Formatos das instruções do PDP-11



Os números abaixo dos campos indicam o tamanho em bits.

Origem e destino contêm cada um campo de modo de endereçamento de 3 bits e o número de registrador de 3 bits.

FP indica um dos quatro registradores de ponto flutuante.

R indica um dos registradores de uso geral.

CC é campo de código condicional.

ser acomodado: normalmente de dois a três. Como algumas operações inerentemente requerem mais operandos, diversas estratégias devem ser empregadas para alcançar o resultado desejado usando duas ou mais instruções.

Para evitar esses problemas, dois critérios foram usados ao projetar o formato de instrução VAX (Strecker, 1978ⁿ):

1. Todas as instruções devem ter um número "natural" de operandos.
2. Todos os operandos devem ter a mesma generalidade na especificação.

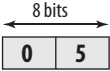
O resultado é um formato de instrução altamente variável. Uma instrução consiste de um *opcode* de 1 ou 2 bytes seguido de especificadores de operando de 0 a 6, dependendo do *opcode*. O tamanho mínimo da instrução é 1 byte e instruções até 37 bytes podem ser construídas. A Figura 11.8 mostra alguns exemplos.

A instrução VAX começa com um *opcode* de 1 byte. Isso é suficiente para lidar com a maioria das instruções VAX. No entanto, como existem mais de 300 instruções diferentes, 8 bits não são suficientes. Os códigos hexadecimais FD e FF indicam um *opcode* estendido, com o *opcode* atual sendo especificado no segundo byte.

O restante da instrução consiste de até seis especificadores de operandos. Um especificador de operando está, no mínimo, num formato de 1 byte onde os quatro bits da esquerda representam o modo de endereçamento. A única exceção a esta regra é o modo literal, o qual é sinalizado com 00 nos dois bits à extrema esquerda, deixando um espaço para um literal de 6 bits. Por causa desta exceção, um total de 12 modos de endereçamento diferentes pode ser especificado.

Um especificador de operando frequentemente consiste de apenas um byte, onde os 4 bits da extrema direita especificam um dos 16 registradores de uso geral. O tamanho do especificador de operando pode ser estendido de duas formas. Primeira: um valor constante de um ou mais bytes podem seguir imediatamente o primeiro byte do especificador de operando. Um exemplo disto é o modo por deslocamento, onde um deslocamento de 8, 16 ou

Figura 11.8 Exemplos de instruções do VAX

Formato hexadecimal	Explicação	Notação Assembler e Descrição
	<i>Opcode</i> para RSB	RSB Retorno da sub-rotina
D 4 5 9	<i>Opcode</i> para CLRL Registrador R9	CLRL R9 Limpar registrador R9
B 0 C 4 6 4 0 1 A B 1 9	<i>Opcode</i> para MOVW Modo de deslocamento da palavra, Registrador R4 356 em hexadecimal Modo de deslocamento de byte Registrador R11 25 em hexadecimal	MOVW 356(R4), 25(R11) Move uma palavra de um endereço que é 356 mais conteúdo de R4 para endereço que é 25 mais conteúdo de R11
C 1 0 5 5 0 4 2 D F	<i>Opcode</i> para ADDL3 Número 5 literal Registrador de modo R0 Índice pré-fixado R2 palavra relativa indireta (deslocamento de PC) Quantidade de deslocamento de relativa à posição A	ADDL3 #5, R0, @A[R2] Adiciona 5 a um inteiro de 32 bits em R0 e armazena o resultado na posição cujo endereço é soma de A e quatro vezes o conteúdo de R2.

32 bits é usado. Segunda: um modo indexado de endereçamento pode ser usado. Neste caso, o primeiro byte do especificador de operando consiste de um código de modo de endereçamento de 4 bits 0100 e de um identificador de um registrador índice de 4 bits. O restante do especificador de operando consiste do especificador básico de endereço, o qual, por sua vez, pode ter um tamanho de um ou mais bytes.

O leitor pode estar se perguntando, conforme o fez o autor, que tipo de instrução requer seis operandos? Surpreendentemente, VAX possui uma série dessas instruções. Considere

ADDP6 OP1, OP2, OP3, OP4, OP5, OP6

Esta instrução soma dois números decimais agrupados. OP1 e OP2 especificam o tamanho e o endereço inicial de uma cadeia de números decimais; OP3 e OP4 especificam uma segunda *string*. Essas duas *strings* são somadas e o resultado é armazenado na cadeia de números decimais cujo tamanho e posição inicial são especificados por OP5 e OP6.

O conjunto de instruções VAX fornece uma grande variedade de operações e modos de endereçamento. Isso dá ao programador, como, por exemplo, um desenvolvedor de compiladores, uma ferramenta para desenvolvimento de programas muito poderosa e flexível. Na teoria, isso deveria levar a compilações eficientes de programas escritos em linguagem de alto nível, para a linguagem de máquina e, em geral, ao uso efetivo e eficiente de recursos do processador. O preço a ser pago por esses benefícios é o aumento da complexidade do processador se comparado a um processador com um conjunto de instruções e formatos mais simples.

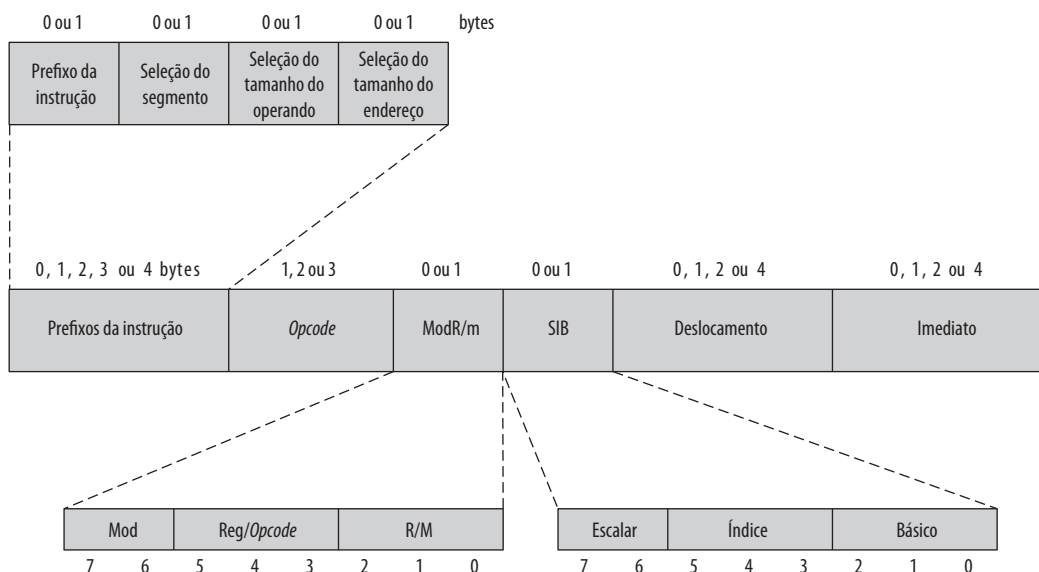
Retornaremos a este assunto no Capítulo 13 onde examinamos uma situação para conjuntos de instruções muito simples.

11.4 Formatos de instruções x86 e ARM

Formatos de instruções x86

O x86 é equipado com uma variedade de conjuntos de instruções. Dos elementos descritos nesta seção apenas o campo *opcode* está sempre presente. A Figura 11.9 ilustra o formato de instrução geral. As instruções são feitas de 0 a 4 prefixos opcionais, um *opcode* de 1 ou 2 bytes, um especificador de endereço opcional (que consiste de byte ModR/m e byte de Índice Escalar), um deslocamento opcional e um campo imediato opcional.

Figura 11.9 Formato da instrução x86



Vamos primeiramente considerar os bytes de prefixo:

- **Prefixos da instrução:** o prefixo da instrução, se estiver presente, consiste do prefixo LOCK ou de um dos prefixos de repetição. O prefixo LOCK é usado para garantir o uso exclusivo da memória compartilhada em ambientes multiprocessados. Os prefixos de repetição especificam operações repetidas de uma *string*, o que possibilita que x86 processe *strings* muito mais rapidamente do que com laços normais de software. Existem cinco prefixos de repetição diferentes: REP, REPE, REPZ, REPNE e REPNZ. Quando o prefixo absoluto REP está presente, a operação especificada na instrução é executada repetidamente em elementos sucessivos da *string*; o número de repetições é especificado no registrador CX. O prefixo REP condicional faz com que a operação seja repetida até que o contador em CX chegue a zero ou até que a condição seja satisfeita.
- **Seleção do segmento:** determina explicitamente qual registrador de segmento que uma instrução deve usar, alterando o registrador do segmento padrão gerado pelo x86 para essa instrução.
- **Tamanho do operando:** uma instrução possui um tamanho padrão de operando de 16 ou 32 bits e o prefixo do operando alterna entre operandos de 32 e 16 bits.
- **Tamanho do endereço:** o processador pode endereçar memória usando endereços de 16 ou 32 bits. O tamanho do endereço determina o tamanho do deslocamento usado nas instruções e o tamanho dos offsets de endereço gerados durante o cálculo do endereço efetivo. Um desses tamanhos é definido como padrão e o prefixo do tamanho do endereço alterna entre geração de endereço de 16 e 32 bits.

A instrução em si inclui os seguintes campos:

- **Opcode:** o campo *opcode* tem o tamanho de 1, 2 ou 3 bytes. O *opcode* pode também incluir bits que especificam se a informação é byte ou tamanho cheio (16 ou 32 bits dependendo do contexto), direção da operação dos dados (para ou de memória) e se um campo de dados imediato deve ser estendido por um sinal.
- **ModR/m:** este byte, e o próximo, fornece o endereçamento da informação. O byte ModR/m especifica se um operando está em um registrador ou na memória; se estiver na memória, então os campos dentro do byte especificam o modo de endereçamento a ser usado. O byte ModR/m consiste de três campos: campo Mod (2 bits) é combinado com o campo r/m para formar 32 valores possíveis: 8 registradores e 24 modos de indexação; o campo Reg/*Opcode* (3 bits) especifica ou um número de registrador ou mais três bits da informação do *opcode*; o campo r/m (3 bits) pode especificar um registrador como sendo a posição de um operando ou pode fazer parte do código do modo de endereçamento em combinação com o campo Mod.
- **SIB:** codificação do byte ModR/m especifica a inclusão do byte SIB para determinar totalmente o modo de endereçamento. O byte SIB consiste de três campos: campo Escalar (2 bits) define o fator de escala para indexação escalar; o campo Índice (3 bits) determina o registrador indexador; campo Base (3 bits) especifica o registrador base.
- **Deslocamento:** quando o especificador do modo de endereçamento indica que o deslocamento é usado, um campo de valor inteiro com sinal para deslocamento de 8, 16 ou 32 bits é adicionado.
- **Imediato:** fornece o valor de um operando de 8, 16 ou 32 bits.

Diversas comparações podem ser úteis aqui. No formato x86, o modo de endereçamento é fornecido como parte da sequência de *opcode* em vez de com cada operando. Como apenas um operando pode ter a informação sobre o modo de endereçamento, apenas um operando de memória pode ser referenciado em uma instrução. Ao contrário disso, o VAX carrega a informação sobre o modo de endereçamento com cada operando, possibilitando operações do tipo memória-para-memória. Por isso, as instruções x86 são mais compactas. No entanto, se uma operação memória-para-memória é necessária, o VAX pode executá-la em uma única instrução.

O formato x86 permite o uso de offset para indexação de não apenas 1 byte, mas também de 2 e 4. Embora o uso de offsets maiores para indexação resulte em instruções maiores, esse recurso permite a flexibilidade necessária. Por exemplo, é útil ao endereçar grandes vetores ou grandes *stackframes*. Ao contrário disso, o formato de instrução de IBM S/370 não permite offsets maiores que 4K bytes (12 bits da informação de offset) e o offset deve ser positivo. Quando uma posição não está ao alcance desse offset, o compilador precisa de código extra para gerar o endereço necessário. Esse problema aparece especialmente quando se lida com *stackframes* que possuem variáveis locais ocupando um espaço maior que 4K bytes. Segundo Dewar (1990), "gerar código para S/370 é tão doloroso por causa dessa restrição que houve até compiladores para S/370 que simplesmente optavam por limitar o tamanho do *stackframe* a 4K bytes".

Como podemos ver, a codificação do conjunto de instruções x86 é muito complexa. Isso tem a ver parcialmente com a necessidade de manter compatibilidade com as máquinas 8086 e parcialmente com o desejo de uma parte dos projetistas para fornecer toda a assistência possível ao projetista de compiladores para produzir um código eficiente. É uma questão a ser discutida se um conjunto de instruções complexo como este é preferível ao extremo oposto, que é o conjunto de instruções RISC.



Formatos de instruções ARM

Todas as instruções na arquitetura ARM têm o tamanho de 32 bits e seguem um formato regular (Figura 11.10). Os quatro primeiros bits da instrução são código condicional. Conforme discutido no Capítulo 10, na teoria todas as instruções ARM podem ser executadas condicionalmente. Os três próximos bits definem o tipo geral da instrução. Para a maioria das instruções, exceto as do tipo *branch*, os próximos cinco bits constituem um *opcode* e/ou bits modificadores para a operação. Os 20 bits restantes são para endereçamento de operando. A estrutura regular dos formatos da instrução facilita o trabalho das unidades de decodificação da instrução.

CONSTANTES IMEDIATAS Para atingir um intervalo maior de valores imediatos, o formato de processamento dos dados imediato especifica o valor imediato e o valor rotacional. O valor imediato de 8 bits é estendido para 32 bits e depois rotacionado para direita por um número de bits igual a duas vezes o valor de rotação de 4 bits. Diversos exemplos são mostrados na Figura 11.11.

CONJUNTO DE INSTRUÇÕES THUMB O conjunto de instruções Thumb é um subconjunto recodificado do conjunto de instruções ARM. O Thumb foi projetado para melhorar o desempenho das implementações ARM que usam um barramento de dados de memória de 16 bits ou limitado e para permitir uma melhor densidade de código do que a fornecida pelo conjunto de instruções ARM. O conjunto de instruções Thumb contém um subconjunto do conjunto de instruções ARM de 32 bits recodificadas para instruções de 16 bits. A economia é obtida da seguinte maneira:

1. As instruções Thumb não são condicionais, então o campo de código da condição não é usado. Além disso, todas as instruções Thumb aritméticas e lógicas atualizam os flags de condição, então o bit para o flag de atualização não é necessário. Economia: 5 bits.

Figura 11.10 Formatos da instrução ARM

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Processamento de dados por deslocamento imediato	Cond	0	0	0	Opcode					S	Rn	Rd	Quant. de deslocamento				Deslocamento				0	Rm										
Processamento de dados por deslocamento de registrador	Cond	0	0	0	Opcode					S	Rn	Rd	Rs				0	Deslocamento				1	Rm									
Processamento de dados imediato	Cond	0	0	1	Opcode					S	Rn	Rd	Rotacionar				Imediato															
Offset imediato para carregar/armazenar	Cond	0	1	0	P	U	B	W	L	Rn	Rd	Imediato																				
Offset de registrador para carregar/armazenar	Cond	0	1	1	P	U	B	W	L	Rn	Rd	Quant. de deslocamento				Deslocamento				0	Rm											
Múltiplo carregar/armazenar	Cond	1	0	0	P	U	S	W	L	Rn	Lista de registradores																					
Condição/condição com link	Cond	1	0	1	L	Offset de 24 bits																										

S = para instruções de processamento de dados, significa que a instrução atualiza o código da condição.

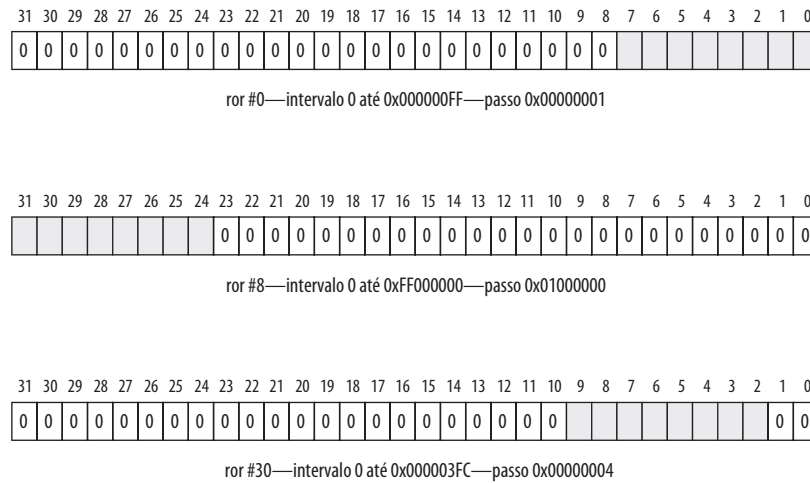
S = para instruções múltiplas de carregar/armazenar, significa se a execução da instrução é restrita ao modo supervisor.

P, U, W = bits usados para distinguir diferentes tipos de modo de endereçamento.

B = diferença entre um byte sem sinal (B == 1) e uma palavra (B == 0).

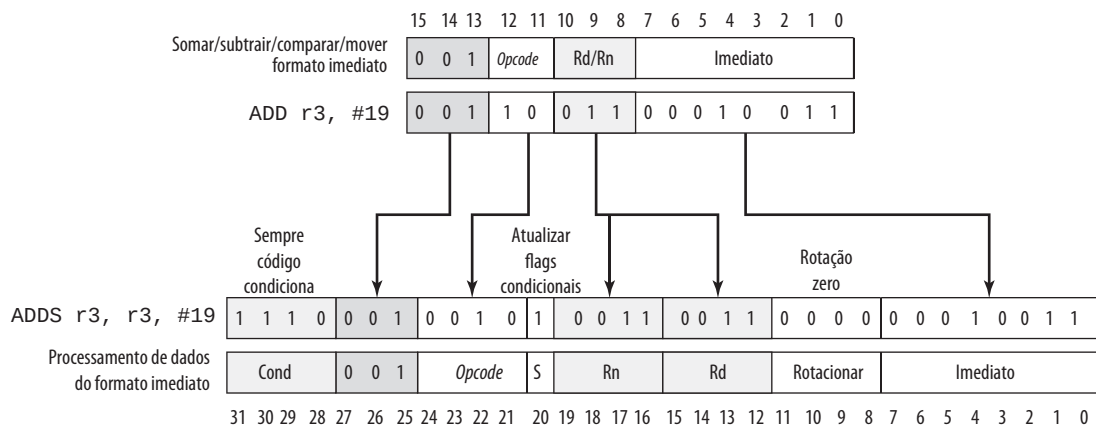
L = para instruções de carregar/armazenar, usado para diferenciar entre carregar (L == 1) e armazenar (L == 0).

L = para instruções condicionais, determina se o endereço de retorno é armazenado no registrador vinculado.

Figura 11.11 Exemplos de uso de contantes imediatas ARM

2. O Thumb possui apenas um subconjunto de operações do conjunto de instruções completo e usa apenas um campo de *opcode* de 2 bits mais um campo de tipo de 3 bits. Economia: 2 bits.
3. A economia restante de 9 bits vem da redução na especificação do operando. Por exemplo, as instruções Thumb referenciam apenas os registradores de r0 até r7, então apenas 3 bits são necessários para referências dos registradores, em vez de 4 bits. Os valores imediatos não incluem um campo rotacional de 4 bits.

O processador ARM pode executar um programa consistindo de uma mistura de instruções Thumb e instruções ARM de 32 bits. Um bit no registrador de controle do processador determina qual tipo de instrução está sendo executada no momento. A Figura 11.12 mostra um exemplo. A figura mostra tanto o formato geral como uma instância específica de uma instrução em ambos os formatos, 16 e 32 bits.

Figura 11.12 Transformando a instrução ADD Thumb na instrução ARM equivalente



11.5 Linguagem de montagem

Um processador pode entender e executar instruções de máquina. Essas instruções são simples números binários armazenados no computador. Se o programador quisesse programar diretamente na linguagem de máquina, então seria necessário informar o programa como dados binários.

Considere a simples instrução BASIC:

$$N = I + J + K$$

Suponha que queiramos programar esta instrução na linguagem de máquina e inicializar I, J e K para 2, 3 e 4, respectivamente. Isso é mostrado na Figura 11.13a. O programa inicia na posição 101 (hexadecimal). Memória é reservada para as quatro variáveis iniciando na posição 201. O programa consiste de quatro instruções:

1. Carrega o conteúdo da posição 201 em AC.
2. Adiciona o conteúdo da posição 202 a AC.
3. Adiciona o conteúdo da posição 203 a AC.
4. Armazena o conteúdo de AC na posição 204.

Este é claramente um processo tedioso e propenso a erros.

Um pequeno avanço seria escrever o programa em hexadecimal no lugar de binário (Figura 10.11b). Podemos escrever o programa como uma série de linhas. Cada linha contém o endereço de uma posição de memória e o código hexadecimal do valor binário para ser armazenado nessa posição. Depois precisamos de um programa que irá aceitar essa entrada, traduzir cada linha em um número binário e armazená-lo em uma posição específica.

Para melhorar mais, podemos usar o nome simbólico ou o mnemônico de cada instrução. Isso resulta em um *programa simbólico* mostrado na Figura 10.11c. Cada linha de entrada ainda representa uma posição de memória. Cada linha consiste de três campos, separados por espaços. O primeiro campo contém o endereço de uma posição. Para uma instrução, o segundo campo contém um símbolo de três letras para *opcode*. Quando se trata de uma instrução que referencia à memória, então o terceiro campo contém o endereço. Para armazenar dados arbitrários em uma

Figura 11.13 Computação na fórmula $N = I + J + K$

Endereço		Conteúdo	
101	0010	0010	101
102	0001	0010	102
103	0001	0010	103
104	0011	0010	104
201	0000	0000	201
202	0000	0000	202
203	0000	0000	203
204	0000	0000	204

(a) Programa binário

Endereço	Conteúdo
101	2201
102	1202
103	1203
104	3204
201	0002
202	0003
203	0004
204	0000

(b) Programa hexadecimal

Endereço	Instrução	
101	LDA	201
102	ADD	202
103	ADD	203
104	STA	204
201	DAT	2
202	DAT	3
203	DAT	4
204	DAT	0

(c) Programa simbólico

Rótulo	Operação	Operando
FORMUL	LDA	I
	ADD	J
	ADD	K
	STA	N
I	DATA	2
J	DATA	3
K	DATA	4
N	DATA	0

(d) Programa *assembly*

posição, inventamos uma *pseudoinstrução* com o símbolo DAT. Isso é apenas uma indicação de que o terceiro campo da linha contém um número hexadecimal para ser armazenado na posição especificada no primeiro campo.

Para este tipo de entrada precisamos de um programa um pouco mais complexo. O programa aceita cada linha de entrada, gera um número binário com base no segundo e terceiro (se estiver presente) campos e o armazena na posição definida no primeiro campo.

A utilização de um programa simbólico torna a vida muito mais fácil, mas ainda é estranho porque precisamos dar um endereço absoluto para cada palavra. Isso significa que o programa e os dados podem ser carregados em apenas uma posição da memória e nós precisamos saber essa posição antecipadamente. Pior ainda, suponha que queiramos mudar o programa um dia adicionando ou excluindo uma linha. Isso irá alterar os endereços de todas as palavras subsequentes.

Um sistema bem melhor, e normalmente utilizado, é usar endereços simbólicos. Isso é ilustrado na Figura 10.11d. Cada linha ainda consiste de três campos. O primeiro campo ainda é para endereço, porém um símbolo é usado no lugar de um endereço numérico absoluto. Algumas linhas não possuem endereço, o que implica que o endereço dessa linha é um a mais do que o endereço da linha anterior. Para instruções que referenciam memória, o terceiro campo também contém um endereço simbólico.

Com estas últimas melhorias nós temos uma *linguagem de montagem* (*linguagem assembly*). Programas escritos em linguagem de montagem (programas *assembly*) são traduzidos para linguagem de máquina por um *assembler* (montador). Esse programa precisa fazer não apenas a tradução simbólica discutida anteriormente, como deve também atribuir endereços de memória para endereços simbólicos.

O desenvolvimento da linguagem de montagem foi um grande marco na evolução da tecnologia de computadores. Foi o primeiro passo para linguagens de alto nível usadas atualmente. Embora poucos programadores usem linguagem de montagem, teoricamente todas as máquinas fornecem uma. Elas são usadas, quando usadas, para programas de sistema como compiladores e rotinas de E/S.

O Apêndice B fornece uma explicação mais detalhada sobre a linguagem de montagem.



11.6 Leitura recomendada

As referências citadas no Capítulo 10 são igualmente aplicáveis ao material deste capítulo. Blaauw e Brooks (1997¹) contém uma discussão detalhada sobre formatos das instruções e modos de endereçamento. Além disso, o leitor pode querer consultar Flynn, Johnson e Wakefield (1985²) que discutem e analisam questões de projeto do conjuntos de instruções, particularmente aquelas relacionadas com formatos.

Principais termos, perguntas de revisão e problemas

Principais termos

Auto-indexação	Endereçamento imediato	Pré-indexação
Endereçamento por registrador base	Indexação	Endereçamento por registradores
Endereçamento direto	Endereçamento indireto	Endereçamento indireto por registradores
Endereçamento por deslocamento	Formato da instrução	Endereçamento relativo
Endereço efetivo	Pós-indexação	Palavra

Perguntas de revisão

- 11.1 Defina resumidamente endereçamento imediato.
- 11.2 Defina resumidamente endereçamento direto.
- 11.3 Defina resumidamente endereçamento indireto.
- 11.4 Defina resumidamente endereçamento de registradores.

- 11.5** Defina resumidamente endereçamento indireto por registradores.
- 11.6** Defina resumidamente endereçamento por deslocamento.
- 11.7** Defina resumidamente endereçamento relativo.
- 11.8** Qual é a vantagem da autoindexação?
- 11.9** Qual é a diferença entre pós-indexação e pré-indexação?
- 11.10** Quais fatores devem ser levados em conta para determinar o uso de bits de endereçamento de uma instrução?
- 11.11** Quais são as vantagens e as desvantagens de usar o formato da instrução de tamanho variável?

Problemas

- 11.1** Dados os seguintes valores de memória e uma máquina de um endereço com um acumulador, quais valores as instruções a seguir carregam no acumulador?
- Palavra 20 contém 40.
 - Palavra 30 contém 50.
 - Palavra 40 contém 60.
 - Palavra 50 contém 70.
- a. CARREGAR IMEDIATO 20
 - b. CARREGAR DIRETO 20
 - c. CARREGAR INDIRETO 20
 - d. CARREGAR IMEDIATO 30
 - e. CARREGAR DIRETO 30
 - f. CARREGAR INDIRETO 30
- 11.2** O endereço armazenado no contador de programa é representado pelo símbolo X1. A instrução armazenada em X1 tem uma parte de endereço (referência de operando) X2. O operando necessário para executar a instrução é armazenado na palavra de memória com endereço X3. Um registrador indexador contém o valor X4. Qual é a relação entre essas diversas grandezas se o modo de endereçamento for (a) direto; (b) indireto; (c) PC relativo; (d) indexado?
- 11.3** Um campo de endereço em uma instrução contém o valor decimal 14. Onde está o operando correspondente para
- a. endereçamento imediato?
 - b. endereçamento direto?
 - c. endereçamento indireto?
 - d. endereçamento de registradores?
 - e. endereçamento indireto de registradores?
- 11.4** Considere um processador de 16 bits no qual aparece dentro da memória principal o seguinte conteúdo, começando na posição 200:

200	Carregar em AC	Modo
201	500	
202	Próxima instrução	

A primeira parte da primeira palavra indica que esta instrução carrega um valor em um acumulador. O campo Modo especifica um modo de endereçamento e, se apropriado, indica um registrador de origem; assuma que, quando usado, o registrador de origem é R1 e tem o valor 400. Há também um registrador base que contém o valor 100. O valor 500 na posição 201 pode ser uma parte do cálculo do endereço. Suponha que a posição 399 contém o valor 999, a posição 400 contém o valor 1.000, e assim por diante. Determine o endereço efetivo e o operando a ser carregado para os seguintes modos de endereçamento:

a. Direto	d. PC relativo	g. Registrador indireto
b. Imediato	e. Deslocamento	h. Autoindexação com incremento usando R1
c. Indireto	f. Registrador	

- 11.5** A instrução de desvio no modo PC-relativo tem o tamanho de 3 bytes. O endereço da instrução, decimal, é 256028. Determine o endereço de destino do desvio se o deslocamento com sinal na instrução for -31 .
- 11.6** A instrução de desvio no modo PC relativo é armazenada na memória no endereço 620_{10} . Um desvio é feito para posição 530_{10} . O campo de endereço da instrução tem o tamanho de 10 bits. Qual é o valor binário na instrução?
- 11.7** Quantas vezes o processador precisa referenciar a memória quando obtém e executa uma instrução no modo de endereçamento indireto se a instrução for (a) um cálculo requerendo um único operando; (b) um desvio?
- 11.8** O IBM 370 não oferece endereçamento indireto. Suponha que o endereço de um operando esteja na memória principal. Como você acessaria o operando?
- 11.9** Em Cook e Dande (1982¹) os autores propõem que o modo de endereçamento PC-relativo seja substituído por outros modos, como o uso de uma pilha. Qual é a desvantagem dessa proposta?
- 11.10** O x86 inclui a seguinte instrução:
- IMUL op1, op2, immediate
- Esta instrução multiplica op2, o qual pode ser registrador ou memória, pelo valor do operando imediato e guarda o resultado em op1, o qual tem que ser um registrador. Não existe nenhuma outra instrução de três operandos deste tipo no conjunto de instruções. Qual o possível uso dessa instrução? (*Dica*: considere a indexação).
- 11.11** Considere um processador que inclui um modo de endereçamento base com indexação. Suponha que uma instrução que emprega esse modo de endereçamento e especifica um deslocamento de 1970 em decimal seja encontrada. Atualmente, o registrador base e o indexador contêm os números decimais 48022 e 8, respectivamente. Qual é o endereço do operando?
- 11.12** $EA = (X) +$ é o endereço efetivo igual ao conteúdo da posição X, com X sendo incrementado em tamanho de uma palavra depois que o endereço efetivo é calculado; $EA = - (X)$ é o endereço efetivo igual ao conteúdo da posição X, com X sendo decrementado em tamanho de uma palavra antes que o endereço efetivo seja calculado; $EA = (X) -$ é o endereço efetivo igual ao conteúdo da posição X, com X sendo decrementado em tamanho de uma palavra depois que o endereço efetivo é calculado. Considere as seguintes instruções, cada uma no formato (Operação, Operando de Origem, Operando de Destino) e o resultado da operação sendo guardado no operando de destino.
- OP X, (X)
 - OP (X), (X) +
 - OP (X) +, (X)
 - OP - (X), (X)
 - OP - (X), (X) +
 - OP (X) +, (X) +
 - OP (X) - , (X)
- Usando X como ponteiro da pilha, qual destas instruções pode pegar os dois primeiros elementos do topo da pilha, executar uma determinada operação (por exemplo, SOMAR origem com destino e armazenar no destino) e colocar o resultado de volta na pilha? Para cada instrução dessas, a pilha cresce em direção à posição 0 da memória ou na direção oposta?
- 11.13** Suponha um processador baseado em pilha que inclui as operações de pilha PUSH e POP. As operações aritméticas envolvem automaticamente um ou dois elementos do topo da pilha. Comece com uma pilha vazia. Quais elementos restam na pilha depois que as instruções a seguir são executadas?
- PUSH 4
PUSH 7
PUSH 8
ADD
PUSH 10
SUB
MUL
- 11.14** Justifique a afirmação de que uma instrução de 32 bits é provavelmente menos do que duas vezes mais útil do que uma instrução de 16 bits.
- 11.15** Por que a decisão da IBM de mudar de 36 bits para 32 bits por palavra foi dolorosa e para quem ela o foi?
- 11.16** Suponha um conjunto de instruções que usa um tamanho fixo de instrução de 16 bits. Especificadores de operandos têm o tamanho de 6 bits.

Existem instruções *K* de dois operandos e instruções *L* de zero operandos. Qual é o número máximo de instruções de um operando que podem ser suportadas?

11.17 Projete um *opcode* de tamanho variável que permita que tudo que está relacionado a seguir seja codificado em uma instrução de 36 bits:

- instruções com dois endereços de 15 bits e um registrador numérico de 3 bits
- instruções com um endereço de 15 bits e um registrador numérico de 3 bits
- instruções sem endereços ou registradores

11.18 Considere os resultados do Problema 10.6. Suponha que *M* seja um endereço de memória de 16 bits e que *X*, *Y* e *Z* sejam ou endereços de 16 bits ou registradores numéricos de 4 bits. A máquina de um endereço usa um acumulador e as máquinas de dois e três endereços possuem 16 registradores e instruções operando em todas as combinações de posições de memória e registradores. Assumindo *opcodes* de 8 bits e tamanhos das instruções que sejam múltiplos de 4 bits, quantos bits cada máquina precisa para calcular *X*?

11.19 Existe alguma justificativa possível para uma instrução com dois *opcodes*?

11.20 O Zilog Z8001 de 16 bits possui o seguinte formato geral da instrução:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Modo		Opcode						w/b	Operando 2				Operando 1		

O campo *modo* define como localizar operandos a partir dos campos *operando*. O campo *w/b* é usado em certas instruções para determinar se os operandos são bytes ou palavras de 16 bits. O campo *operando 1* pode (dependendo do conteúdo do campo *modo*) definir um dos 16 registradores de uso geral. O campo *operando 2* pode definir qualquer registrador de uso geral exceto registrador 0. Quando o campo *operando 2* for todo zero, cada um dos *opcodes* originais assume um novo significado.

- a. Quantos *opcodes* são fornecidos em Z8001?
- b. Sugira uma maneira eficiente para prover mais *opcodes* e indique a negociação envolvida.

Referências

- a. FRAILEY, D. "Word length of a computer architecture: definitions and applications". *Computer Architecture News*, jun. 1983.
- b. LUNDE, A. "Empirical evaluation of some features of instruction set processor architectures". *Communications of the ACM*, mar. 1977.
- c. HUCK, T. *Comparative analysis of computer architectures*. Stanford University Technical Report No. 83-243, mai. 1983.
- d. CRAGON, H. "An evaluation of code space requirements and performance of various architectures". *Computer Architecture News*, fev. 1979.
- e. BELL, C.; NEWELL, A. e SIEWIOREK, D. "Structural levels of the PDP-8". Em: BELL, C; MUDGE, J. e McNAMARA, J. *Computer engineering: a DEC view of hardware systems design*. Bedford, MA: Digital Press, 1978.
- f. BELL, C.; KOTOK, A. HASTINGS, T. e HILL, R. "The evolution of the DEC system-10". *Communications of the ACM*, jan. 1978.
- g. BELL, C; CADY, R.; McFARLAND, H.; DELAGI, B.; O'LOUGHLIN, J. e NOONAN, R. "A new architecture for minicomputers—the DEC PDP-11". *Proceedings, Spring Joint Computer Conference*, 1970.
- h. STRECKER, W. "VAX-11/780: a virtual address extension to the DEC PDP-11 family". *Proceedings, National Computer Conference*, 1978.
- i. DEWAR, R. e SMOSNA, M. *Microprocessors: a programmer's view*. Nova York: McGraw-Hill, 1990.
- j. BLAAUW, G. e BROOKS, F. *Computer architecture: concepts and evolution*. Reading, MA: Addison-Wesley, 1997.
- k. FLYNN, M.; JOHNSON, J.; e WAKEFIELD, S. "On instruction sets and their formats." *IEEE Transactions on Computers*, mar. 1985.
- l. COOK, R. e DANDE, N. "An experiment to improve operand addressing". *Proceedings, Symposium on Architecture Support for Programming Languages and Operating Systems*, mar. 1982.